University of Pretoria · Department of Computer Science

COS750 Educational Software Development

COS212 DFS Tutor

Depth-First Search on a Binary Search Tree

Instructional Design Document (ID Document)

Version	1.0
Date	28 October 2026
Status	Draft for team review
Companion document	SE Document v1.1 (requirements and architecture)
Group	_____

This document describes the educational design of the COS212 DFS Tutor: who it is for, what students should learn, how the content is structured and presented, how students are kept motivated, how their progress is assessed and supported, and how we will evaluate whether the app works. The use of generative AI is acknowledged in Appendix E.

Contents

1 Introduction **3**

1.1 Purpose of this document . . . 3

1.2 The brief in one paragraph . . . 3

1.3 How to read this document . . . 3

2 Instructional design approach **3**

2.1 Why ADDIE . . . 3

3 Analysis **4**

3.1 Learner analysis . . . 4

3.2 Context analysis . . . 5

3.3 Content analysis: a learning hierarchy . . . 5

3.4 Misconception analysis . . . 5

3.5 Needs summary . . . 6

4 Learning outcomes and alignment **6**

4.1 Learning outcomes . . . 6

4.2 Constructive alignment . . . 7

5 Lesson structure **7**

5.1 Gagné's Nine Events, screen by screen . . . 7

5.2 Learning journey . . . 8

5.3 Sequencing principles . . . 9

5.4 Check against Merrill's First Principles . . . 10

6 Learning theories and how we apply them **10**

6.1 Behaviourism: practice with immediate feedback . . . 10

6.2 Cognitivism: managing cognitive load . . . 10

6.3 Constructivism: predict, observe, explain . . . 11

6.4 Social constructivism: scaffolding in the zone of proximal development . . . 11

7 Presentation design **11**

7.1 Visual language . . . 11

7.2 Writing style . . . 11

7.3 Storyboards . . . 12

8 Catering for different learners **13**

8.1 Learning styles: what we do and what we do not claim . . . 13

8.2 Universal Design for Learning . . . 13

8.3 Different starting points . . . 14

9 Motivation **14**

10 Assessment, feedback and scaffolding **15**

10.1 Purpose of assessment in a pre-lecture app 15

10.2 The readiness quiz 15

10.3 Feedback design 15

10.4 Hint ladder in guided practice 16

10.5 Scaffolding after the quiz 16

11 Development **17**

11.1 Content production 17

11.2 Example trees 17

11.3 Trace of the practice search 18

12 Implementation **18**

12.1 Delivery to students 18

12.2 Guidance for the lecturer 18

12.3 Examination demonstration 19

13 Evaluation **19**

13.1 Evaluation questions 19

13.2 Formative evaluation plan 19

13.3 Instruments 19

13.4 Success criteria 20

14 Reflection on our instructional choices **20**

A Lesson script **22**

B Quiz bank **24**

C Glossary **28**

D Evaluation instruments **28**

E Acknowledgement of generative AI use **29**

1 Introduction

1.1 Purpose of this document

The COS212 lecturer wants students to arrive at the lecture on depth-first search having already met the idea once, in a low-pressure setting, with something like a tutor beside them. Our answer is the COS212 DFS Tutor: a short, self-paced web app that teaches depth-first search (DFS) on a binary search tree (BST), lets students practise it, checks whether they are ready, and helps them if they are not.

The SE document describes how the app is built. This document describes how it *teaches*. It sets out the instructional design approach we followed, the analysis behind our decisions, the learning outcomes, the learning theories we drew on, how the app caters for different learners and keeps them motivated, how assessment and scaffolding work, and how we plan to evaluate the result. Section 14 reflects honestly on the choices we made and their limits.

1.2 The brief in one paragraph

The examination specification asks for a prototype that teaches one core data structure and one core algorithm from CS2023 [7] to students who *have not yet seen them*. The app must include some form of assessment that can be used to give students additional scaffolding before the lecture. We chose the binary search tree and depth-first search. The full session is designed to take about 20 to 25 minutes.

1.3 How to read this document

The document follows the five phases of our instructional design approach, ADDIE: Analysis (Section 3), Design (Sections 4–10), Development (Section 11), Implementation (Section 12) and Evaluation (Section 13). The complete learner-facing lesson text, the full quiz bank, a glossary and our evaluation instruments are in the appendices, so that a reader can see exactly what a student will see.

2 Instructional design approach

2.1 Why ADDIE

We use **ADDIE** (Analysis, Design, Development, Implementation, Evaluation) [4] as our instructional design approach. We considered three alternatives:

- **SAM** (Successive Approximation Model) suits teams that can run many quick prototype-and-test rounds with real learners. We have two weeks and limited access to students who have not done COS212 yet, so we cannot iterate that often.
- **Dick and Carey** is thorough but heavy for a short prototype; several of its ten steps (for example a full performance context analysis) would cost more time than they return here.
- **Merrill's First Principles** [15] describe what good instruction contains rather than how to run a project. We use them as a check on our design (Section 5.4), not as the process.

ADDIE fits our situation best. It is easy for every group member to follow, it maps neatly onto the two-week plan in the SE document, and it has a clear place for evaluation. We use it in its modern, iterative form: evaluation is not a final step but runs alongside every phase, as Figure 1 shows.

Inside the Design phase we need a way to sequence a single lesson. For that we use **Gagné's Nine Events of Instruction** [9]. ADDIE tells us how to run the project; Gagné tells us what order the screens should come in. The two work at different levels and do not compete.

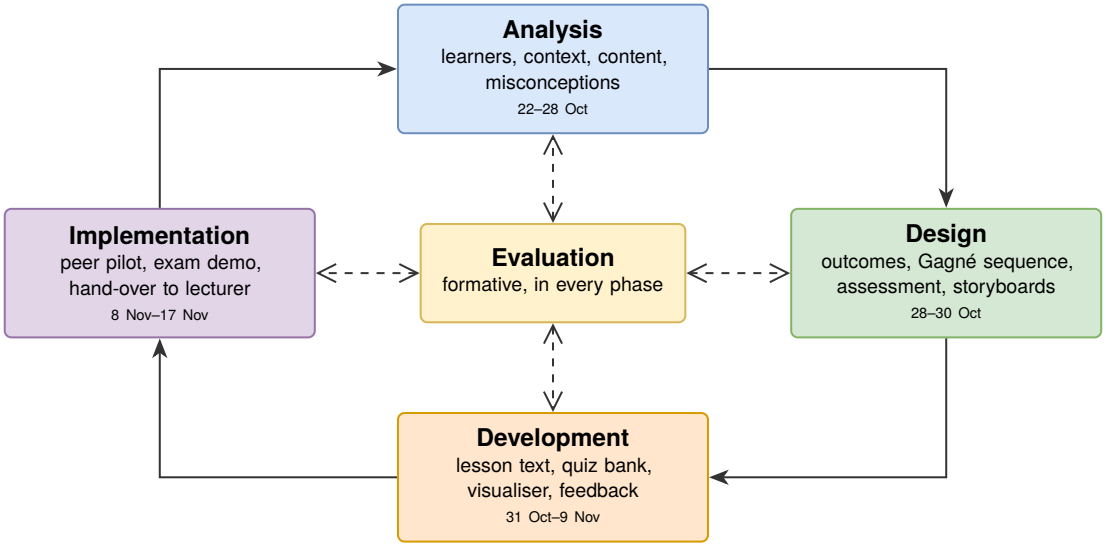


Figure 1: ADDIE as we apply it, with evaluation at the centre of every phase.

3 Analysis

3.1 Learner analysis

Our learners are second-year Computer Science students registered for COS212 at the University of Pretoria. What we can reasonably assume about them:

Characteristic	What it means for the design
Completed first-year programming	They can read simple Java-like code and know what an `if` statement and a loop are. We may show code, but never rely on it.
No prior knowledge of BSTs or DFS	The specification is explicit about this. Every term is introduced before it is used, and the warm-up check catches students who do not know what a tree or a stack is.
Mixed confidence with recursion	Some students find recursion frightening. We teach DFS with an explicit stack first, and only show the recursive version as an optional extra.
Busy, self-directed, time-poor	The session must be short (about 20–25 minutes), work in one sitting, and make progress visible so they know how much is left.
Studying on their own laptops	Laptop browser is the target platform; phones are out of scope for the prototype.
Diverse language backgrounds	Many students study in their second or third language. We use short sentences, everyday words and a glossary, and avoid idioms where possible.
Varied access needs	Some students are colour-blind or use the keyboard more than the mouse. Colour is never the only signal and everything works from the keyboard.

Because the app is used *before* the lecture, it carries no marks. Motivation therefore has to come from the experience itself and from students seeing that it will make the lecture easier (Section 9).

3.2 Context analysis

- **When:** two or three days before the COS212 lecture on tree traversal.
- **Where:** at home, in a lab or in residence, on a laptop browser, alone.
- **How long:** one sitting of about 20–25 minutes; the student can stop at any point.
- **Support available:** none in the moment. The app has to do the tutor's job: explain, check, hint and encourage.
- **Stakes:** none. The quiz is for the student's own information. Nothing is recorded or sent to the lecturer in this prototype.

3.3 Content analysis: a learning hierarchy

Following Gagné's idea of a learning hierarchy, we broke the target skill ("trace a DFS on a BST") into the smaller skills it depends on. Figure 2 shows the result. Anything below the dashed line is assumed entry behaviour; everything above it is taught by the app. The hierarchy directly determines the order of the lesson: we teach from the bottom up.

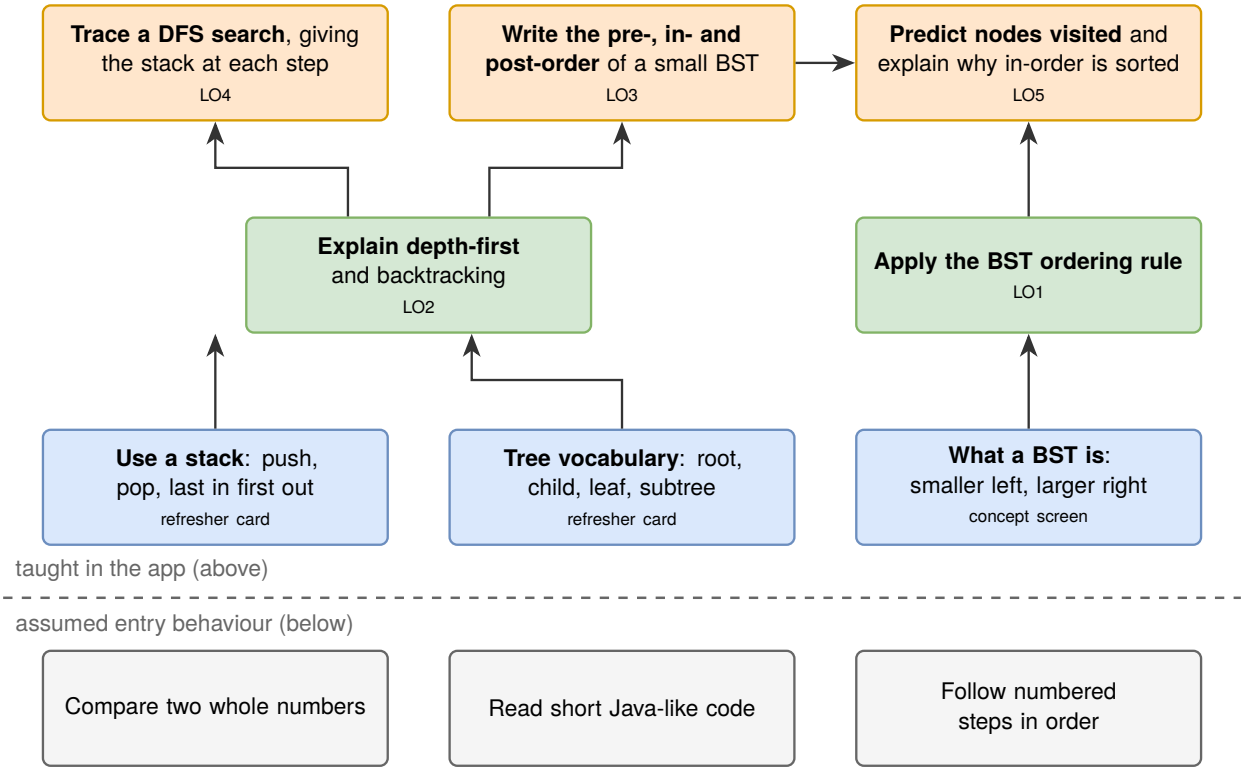


Figure 2: Learning hierarchy for the session. Arrows point from a prerequisite to the skill that depends on it.

3.4 Misconception analysis

Teaching works best when it is aimed at what students actually get wrong. From the literature on algorithm visualisation and our own experience of learning this topic, we identified five likely misconceptions. The COS212 tutor in our expert review (Section 13) will be asked to confirm or

add to this list. Each one is targeted at least three times in the app: once in the explanation, once in guided practice and once in the quiz.

ID	Misconception	What it looks like	How the app responds
M1	Depth-first means level by level	After 50 the student picks 70, the other child of the root	Maze analogy; step player shows the walk going *down*; quiz distractor with targeted feedback
M2	The stack is invisible or optional	Student cannot say what happens after a leaf	Stack panel is always visible; "what is on top?" is the first hint
M3	Pre-, in- and post-order are different walks	Student re-traces the tree from scratch for each order	"Three orders, one walk" screen; the code view shows only one line moving
M4	DFS uses the BST ordering to choose a direction	Student says DFS finds 80 in 3 steps	Explicit "what DFS does *not* do" screen; comparison on the summary screen
M5	Pushing left first makes left come out first	Student writes the stack as [70, 30] after visiting 50	Plate analogy ("last on, first off"); stack animation shows the order of pushes

3.5 Needs summary

Students need to (1) form a correct mental model of "go deep, then back up", (2) see the stack that makes backtracking possible, (3) practise with feedback until the steps feel routine, and (4) know honestly whether they are ready for the lecture. These four needs drive everything in the Design phase.

4 Learning outcomes and alignment

4.1 Learning outcomes

Each outcome is written so that it can be observed and checked, and is tied to a level of the revised Bloom's taxonomy [1]. The audience for all outcomes is a COS212 student working alone with the app.

ID	Bloom level	By the end of the session, the student can...
LO1	Remember	state the BST ordering rule and name the data structure DFS uses to remember where to go back to (a stack), choosing correctly from four options.
LO2	Understand	explain in their own words what "depth-first" and "backtracking" mean, and identify the next node DFS visits from a given point in a tree.
LO3	Apply	write the pre-, in- and post-order of a BST with up to seven nodes, without errors.
LO4	Apply	trace a DFS search for a value on a BST with up to seven nodes, giving the stack contents after each step.

ID	Bloom level	By the end of the session, the student can...
LO5	Analyse	predict how many nodes DFS visits for a given target, explain why it may visit more nodes than necessary, and explain why the in-order of a BST is always sorted.

We deliberately stop at *Analyse*. Evaluating DFS against other search strategies is the job of the lecture. The app ends by handing that question over (Section 5).

4.2 Constructive alignment

Biggs' principle of constructive alignment [3] says that outcomes, learning activities and assessment should all point at the same thing. Table 4 shows how each outcome is taught, practised and assessed.

LO	Taught in	Practised in	Assessed by
LO1	Concept screens 4a and 4c; refresher cards	Warm-up check; inserting values into the tree	Quiz slot 1 (stack or BST insertion)
LO2	Concept screen 4b (maze); worked example narration	Predict-the-next-node in practice 1	Quiz slot 2 (next node or backtracking)
LO3	Screen 4e "three orders, one walk"	Practice 2 (in-order of tree B)	Quiz slot 3 (ordering task)
LO4	Worked example with stack panel	Practice 1 with stack prediction	Quiz slot 4 (stack contents)
LO5	Screens 4e and 4f (sorted in-order; what DFS does not do)	Summary comparison	Quiz slots 5 and 6

Table 4: Constructive alignment of outcomes, activities and assessment.

5 Lesson structure

5.1 Gagné's Nine Events, screen by screen

The session is a sequence of nine events [9]. Table 5 describes what happens at each one and why. The full student-facing text for every screen is in Appendix A.

Event	What the student experiences	Design reason
1. Gain attention	A small maze with three forks. "You can only see the passage in front of you. How would you make sure you never miss the exit?" The student picks a strategy.	A puzzle creates curiosity and an everyday anchor for the idea of going deep and backing up.
2. Inform learners of objectives	Three goals in plain language, plus "this takes about 20 minutes".	Reduces anxiety and lets students judge their own progress.

Event	What the student experiences	Design reason
3. Stimulate recall of prior learning	Two quick questions: "Which picture is a tree?" and "Push 1, 2, 3 onto a stack. What comes off first?" A wrong answer opens a short refresher card before continuing.	Activates and checks prerequisites from the learning hierarchy without making confident students sit through revision.
4. Present the content	Six short concept screens (4a–4f): BSTs, the maze idea, the stack, the four DFS steps, three orders in one walk, and what DFS does not do.	One idea per screen keeps cognitive load low (segmenting and pre-training).
5. Provide learning guidance	A fully worked example: DFS search for 60 on tree A, one step per click, with narration, highlighted nodes and the stack panel.	Worked examples are the most efficient way for novices to build a schema [2].
6. Elicit performance	Practice 1: the student predicts each next node and the stack on a new tree (tree B). Practice 2: they write the in-order of tree B themselves.	Fading: support is withdrawn gradually, moving from studying to doing.
7. Provide feedback	Every prediction and answer gets immediate, specific feedback, with a three-level hint ladder when they are stuck.	Feedback is most useful straight after the attempt, when the student's reasoning is still fresh.
8. Assess performance	Six-question readiness quiz with a confidence rating on each answer.	Gives the student an honest picture and decides whether to offer guided review.
9. Enhance retention and transfer	Summary of key ideas, a "build your own tree" sandbox and two open questions to take into the lecture.	Encourages generalisation and connects the session to what comes next.

Table 5: The nine events as implemented in the app.

5.2 Learning journey

Figure 3 shows the path through the app, including the optional refresher and the remediation loop. Students can move backwards at any time; only the quiz has a fixed order.

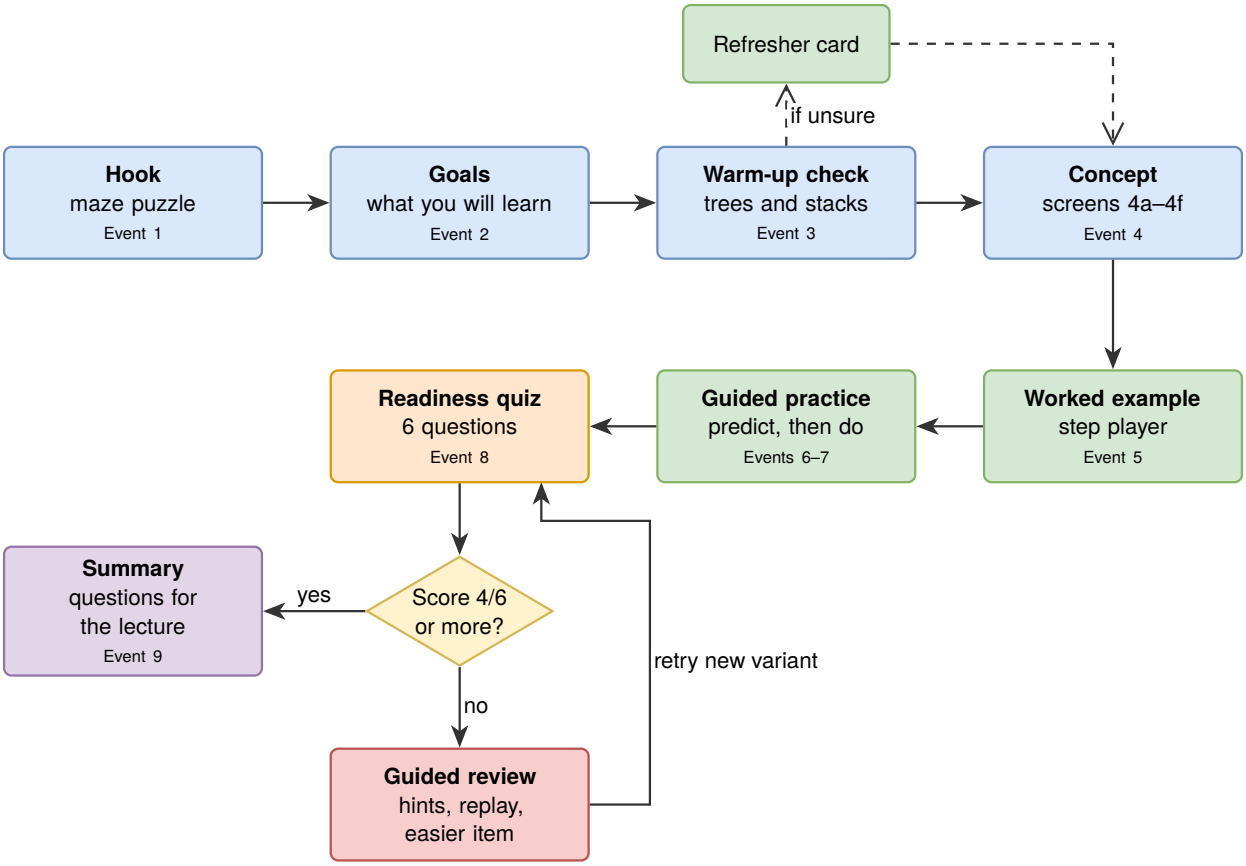


Figure 3: Student learning journey. Blue: instruction; green: guided practice; orange: assessment; red: remediation; purple: consolidation.

5.3 Sequencing principles

- **Bottom-up through the hierarchy.** Vocabulary and the stack come before DFS; DFS search comes before the three traversal orders.
- **Concrete before abstract.** The maze and the plates come before the tree; the tree comes before the code; the code is optional.
- **Iterative before recursive.** The explicit stack makes backtracking visible. The recursive version is offered afterwards as "the same thing, with Java keeping the stack for you".
- **Same tree, then new tree.** The worked example uses tree A; practice uses tree B, which has a different shape (a node with only one child), so students cannot simply memorise a sequence.
- **End with an open question.** The last screen points forward to the lecture rather than closing the topic.

5.4 Check against Merrill's First Principles

Principle	Where it appears
Problem-centred	The session is framed around a real task: finding a value in a tree.
Activation	Maze hook and warm-up check (Events 1 and 3).
Demonstration	Worked example in the step player (Event 5).
Application	Predict mode, the in-order task and the quiz (Events 6 and 8).
Integration	"Build your own tree" sandbox and questions for the lecture (Event 9).

6 Learning theories and how we apply them

We did not pick a single theory and force everything through it. Different parts of the session call for different kinds of learning, so we use each theory where it fits best.

6.1 Behaviourism: practice with immediate feedback

Recalling the BST rule and the stack's last-in-first-out behaviour are small, well-defined responses. For these we use classic behaviourist tools: short questions, immediate confirmation or correction, and a chance to try again. The warm-up check and the first quiz slot work this way.

6.2 Cognitivism: managing cognitive load

DFS asks a beginner to keep track of the tree, the current node, the stack and the output order all at once, which can easily overload working memory. Cognitive load theory [19] and Mayer's multimedia principles [14] shape the presentation:

Principle	Application in the app
Segmenting	One step per click; the student controls the pace; nothing plays automatically.
Pre-training	Vocabulary (root, leaf, stack, push, pop) is taught before the algorithm.
Signalling	Colour plus shape shows each node's state: current, visited, on the stack, found.
Spatial contiguity	The explanation sits directly under the tree, and the stack panel directly beside it, so the eye does not have to search.
Coherence	No decorative images, sounds or animations that do not carry meaning.
Redundancy	Narration is text by default. Read-aloud is optional and does not play at the same time as on-screen text unless the student switches it on.
Personalisation	Conversational, second-person language ("you") rather than formal prose.
Worked-example effect	A full worked example before any problem solving, followed by faded practice [2].

6.3 Constructivism: predict, observe, explain

Students build understanding by testing their own ideas, not just by receiving ours. In predict mode the student commits to an answer *before* the app reveals the next step. When the prediction is wrong, the gap between what they expected and what happened is the moment learning happens. This follows the predict–observe–explain pattern [21]. The "build your own tree" sandbox extends this: students insert their own values and check whether DFS behaves as they expect.

Research on algorithm visualisation supports this choice. A meta-study by Hundhausen, Douglas and Stasko [11] found that *how* students engage with a visualisation matters more than how polished it is, and the engagement taxonomy of Naps et al. [16] distinguishes *viewing* from *responding*, *changing* and *constructing*. The worked example is at the viewing level, predict mode and the quiz are at the responding level, and the sandbox is at the changing level.

6.4 Social constructivism: scaffolding in the zone of proximal development

Vygotsky's zone of proximal development [20] describes what a learner can do with help but not yet alone. Scaffolding [22] is the help that bridges that gap and is then taken away. A real tutor would watch the student, give a small nudge when they are stuck, give more help if that is not enough, and step back as soon as they cope. The app imitates this with the hint ladder in Section 10.4 and with the guided review after the quiz. Support is always offered, never forced, and it fades as the student succeeds.

7 Presentation design

7.1 Visual language

The same visual language is used on every screen, so that students only have to learn it once.

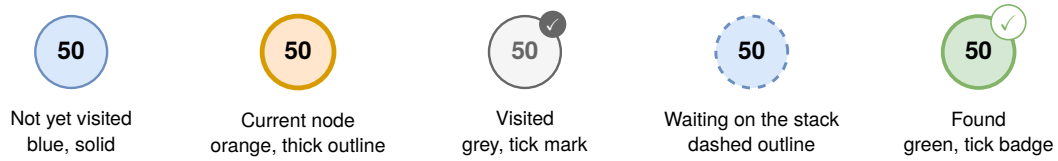


Figure 4: Node states. Each state differs in shape or outline as well as colour, so colour-blind students can tell them apart.

Colours come from a palette that remains distinguishable under the common forms of colour blindness. Text meets WCAG AA contrast, and every interactive control can be reached with the Tab key and used with Enter or Space. Ordering tasks that use drag-and-drop also accept typed input (for example "20, 30, 40") so that keyboard users are not disadvantaged.

7.2 Writing style

All learner-facing text follows the same rules:

- One idea per screen; no more than about 60 words of explanation at a time.
- Short sentences in plain English; technical terms in bold the first time, with a glossary tooltip (Appendix C).
- Second person and active voice: "You visit 30" rather than "Node 30 is visited".

- Explanations say *why*, not just *what*: “30 is on top of the stack, so it comes off next.”
- Feedback never blames: “Not quite” rather than “Wrong”; mistakes are treated as normal.

7.3 Storyboards

Figures 5 and 6 are low-fidelity wireframes of the two most important screens. They are design targets, not final artwork.

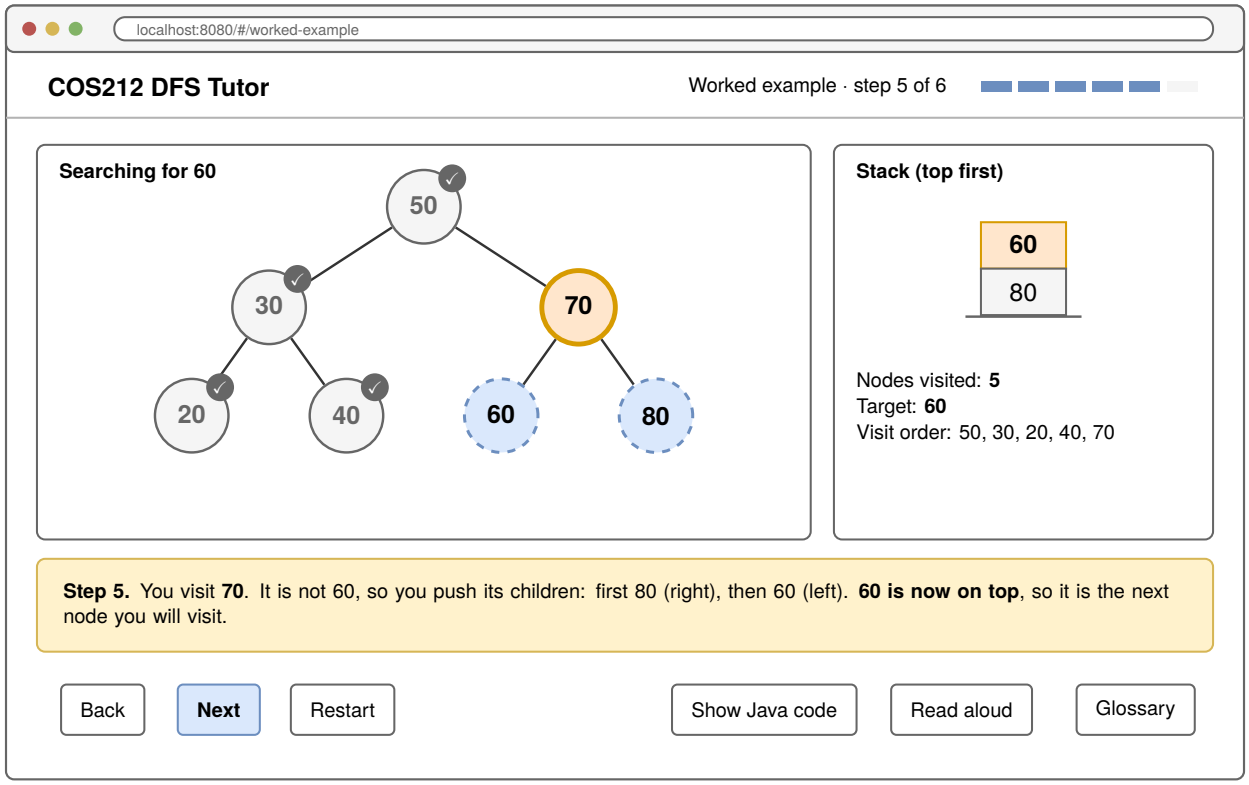


Figure 5: Wireframe of the step player during the worked example (step 5 of the search for 60).

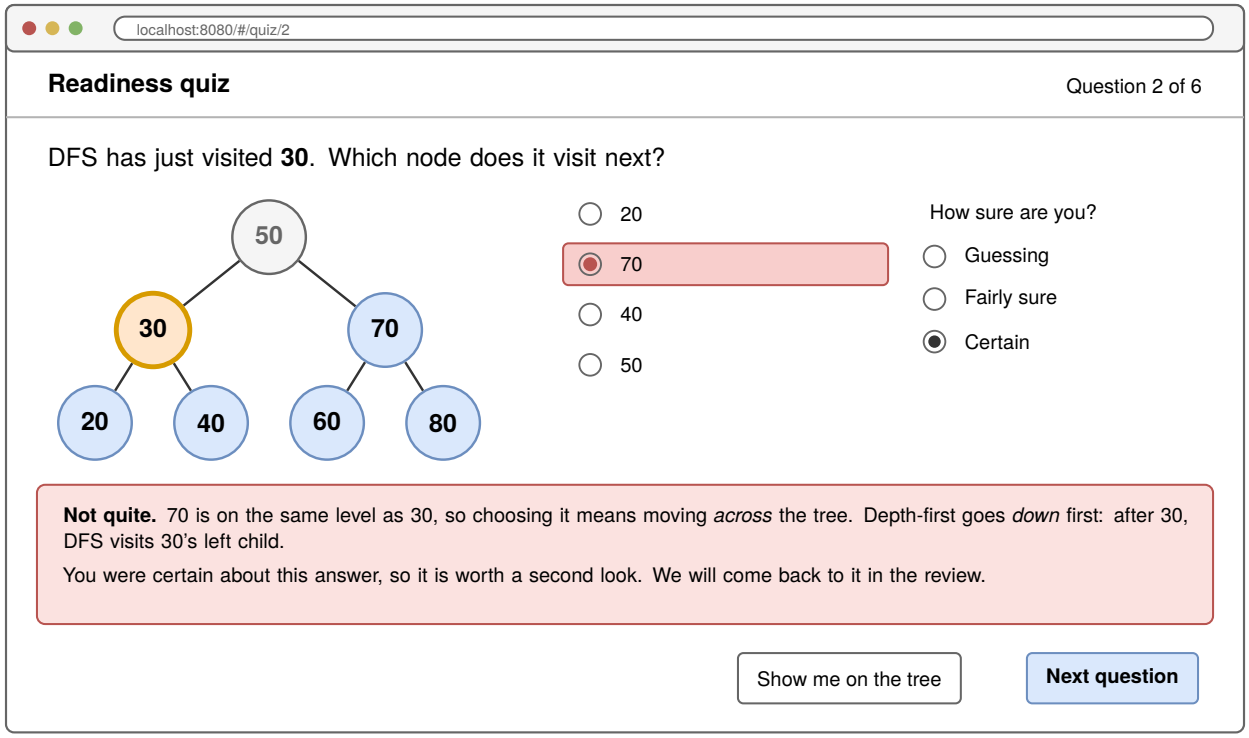


Figure 6: Wireframe of a quiz question after a confident wrong answer (misconception M1).

8 Catering for different learners

8.1 Learning styles: what we do and what we do not claim

The specification asks which learning styles the app caters for. The best known model is VARK (Visual, Aural, Read/write, Kinaesthetic) [8], and we map our features onto it below. However, we want to be careful. A widely cited review by Pashler et al. [17] found little evidence that teaching to a student's *preferred* style improves learning. Our design therefore does not try to diagnose a student's style and adapt to it. Instead, every student gets several representations of the same idea, which research on multimedia learning does support [14]. Students who prefer one mode will still find it, and all students benefit from seeing the idea more than one way.

VARK mode	How the app provides it
Visual	Tree diagram with colour and shape states; animated stack panel; maze picture on the hook screen; side-by-side comparison of the three orders.
Aural	Optional read-aloud of each explanation using the browser's built-in speech synthesis.
Read/write	Short written explanations on every step; glossary tooltips; summary screen that can be printed; optional Java code view.
Kinaesthetic	Clicking through steps at their own pace; predicting the next node; dragging nodes into order; inserting values and watching the tree grow.

8.2 Universal Design for Learning

We also check the design against the three principles of Universal Design for Learning [6], which are about designing for the whole range of learners from the start:

- **Multiple means of engagement:** the student chooses the pace, can skip refresher material they do not need, and can explore their own trees in the sandbox.
- **Multiple means of representation:** picture, stack, text, optional audio and optional code, as above.
- **Multiple means of action and expression:** answers can be given by clicking a node, choosing an option, dragging or typing.

8.3 Different starting points

Some students will already know a little about trees; others will not know what a stack is. The warm-up check handles this. A student who answers both warm-up questions correctly goes straight on; a student who misses one sees the matching refresher card (about one minute each). This keeps the session short for students who are ready while still honouring the requirement that no prior knowledge is assumed.

9 Motivation

Because the app carries no marks, students need reasons of their own to start it and finish it. We use Keller's ARCS model [12] to design for this deliberately.

ARCS	Tactic	Where in the app
Attention	Open with a puzzle rather than a definition.	Hook screen (maze).
	Create small moments of suspense: "What do you think comes next?"	Predict mode.
	Vary the activity every few minutes (read, watch, predict, drag, answer).	Whole journey.
Relevance	Connect to familiar things: folders inside folders, exploring a maze, the undo button (a stack).	Concept screens 4b and 4c.
	Say plainly how this helps: "The lecture will build on this. Doing it now makes the lecture easier to follow."	Goals screen.
Confidence	Clear goals and a visible progress bar.	Every screen.
	Early, easy wins before harder tasks.	Warm-up, then worked example, then practice.
	Hints instead of dead ends; mistakes framed as normal.	Hint ladder, feedback wording.
	Honest, low-stakes readiness check.	Quiz with no marks recorded.
Satisfaction	Immediate feedback that recognises correct reasoning, not just the right answer.	Feedback panel.
	A genuine "aha" moment: the in-order output is sorted.	Screen 4e.
	A summary of what they can now do, and questions to take to the lecture.	Summary screen.

We considered points, badges and leaderboards and decided against them for this prototype. Extrinsic rewards can crowd out interest in the material itself, a leaderboard would require accounts and stored data, and both would take development time away from feedback quality, which matters more for learning.

10 Assessment, feedback and scaffolding

10.1 Purpose of assessment in a pre-lecture app

All assessment in the app is **formative**: its job is to help the student learn and to give them an honest picture of their readiness, not to grade them. There are three kinds:

Type	When	Purpose
Diagnostic	Warm-up check (Event 3)	Find missing prerequisites and route to a refresher.
Embedded	Predict mode and practice (Event 6)	Practise with immediate feedback and hints.
Summative-style	Readiness quiz (Event 8)	Decide whether to offer guided review. Nothing is recorded.

10.2 The readiness quiz

The quiz has six *slots*, each tied to a learning outcome. Every slot has at least two equivalent items (variant A and variant B) in the bank, and a third variant can be generated by relabelling the values of the tree while keeping its shape, with the correct answers calculated by the Java backend. A first attempt draws variant A; a retry draws a variant the student has not seen. The full bank, with every option and its feedback, is in Appendix B.

Slot	LO	Bloom	Item format
1	LO1	Remember	Multiple choice
2	LO2	Understand	Click the node on the tree, or multiple choice
3	LO3	Apply	Put the nodes in order (drag or type)
4	LO4	Apply	Multiple choice about stack contents
5	LO5	Analyse	Multiple choice: how many nodes are visited
6	LO5	Analyse	Multiple choice: explain why

Wrong options are not random. Each one is built from a specific misconception (M1–M5), so the feedback can explain the actual error in the student's thinking.

Confidence rating. After each answer the student says how sure they were: guessing, fairly sure or certain. This does not change the score. It helps students reflect on what they really know, and it helps the app prioritise: a *confident wrong* answer suggests a firmly held misconception and is reviewed first; a *guessed right* answer is flagged as "worth another look" in the summary.

10.3 Feedback design

Feedback is written to answer the three questions Hattie and Timperley identify as making feedback effective [10]: *Where am I going? How am I going? Where to next?* Shute's review of formative

feedback [18] adds practical guidance that we follow: be specific, focus on the task rather than the person, keep it short, and give it immediately for novices.

Every feedback message therefore has the same three parts:

1. **Verdict:** "Correct" or "Not quite".
2. **Reason:** why the answer is right, or which idea led to the wrong answer.
3. **Next move:** what to look at, or what to try next time.

Correct answers get feedback too. A student may be right for the wrong reason, so the reason is always stated.

10.4 Hint ladder in guided practice

When a student makes a wrong prediction in practice, the app does not simply give the answer. It climbs a three-step ladder, giving only as much help as is needed (Figure 7).

Level	Type	Example (predicting the node after 50)
1	Attention nudge	"Look at the stack. Which node is on top?"
2	Rule reminder	"DFS always takes the top node off the stack. After 50 we pushed 70, then 30. Which went on last?"
3	Worked answer	"The next node is 30. It was pushed last, so it is on top and comes off first. That is why DFS explores the left side before the right."

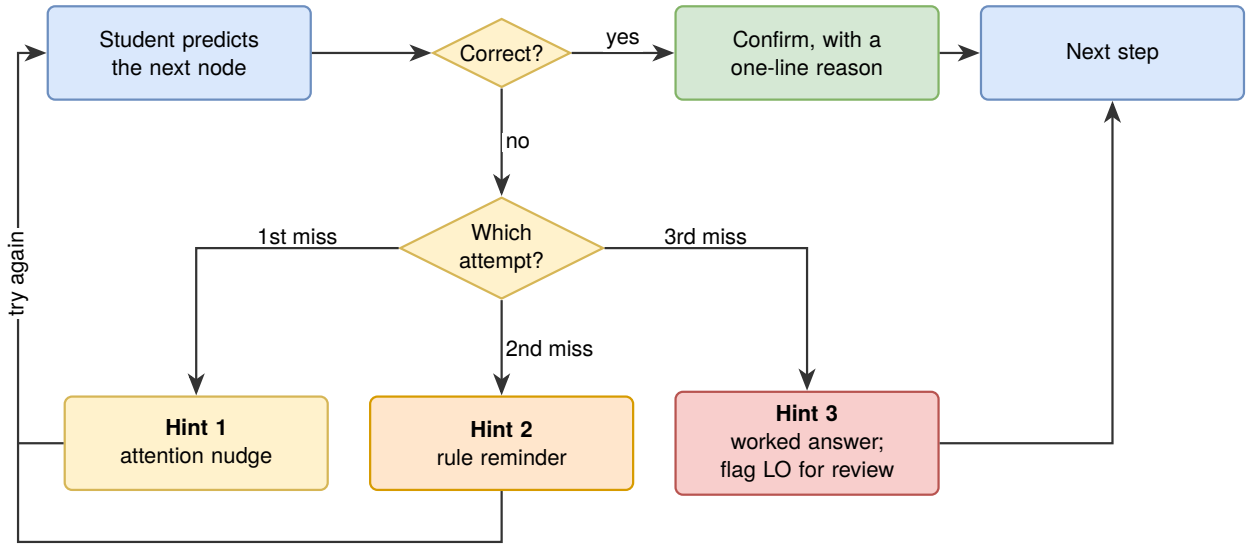


Figure 7: How the app responds to a prediction in guided practice.

If a student reaches hint 3 twice in the same practice, the app suggests replaying the worked example before continuing. It does not insist.

10.5 Scaffolding after the quiz

The quiz outcome decides what is *offered* next:

- **4/6 or more:** the student goes to the summary screen. Any item they got wrong, or guessed right, is listed with a link to the relevant screen.

- **Below 4/6:** the app offers a guided review built from the outcomes they missed, confident wrong answers first. For each missed outcome it (1) replays the relevant part of the worked example with the reason highlighted, (2) asks an easier version of the question with hint 1 already shown, and (3) explains the answer. The student then takes a fresh variant of the quiz.

The student can always choose "Continue to summary anyway". They are adults managing their own preparation, and a forced loop would hurt motivation more than it helps.

Why 4/6? Two correct answers in six could be luck, and slots 5 and 6 are the hardest. A score of 4 or more means the student has at least shown the core Apply-level skills. This is a starting value; Section 13 explains how we will check it.

11 Development

11.1 Content production

1. Lesson text and quiz items are written in JSON files (`lesson.json`, `quiz-bank.json`) so that ID work and code can proceed in parallel.
2. Every quiz answer and every number quoted in a lesson (visit orders, stack contents, node counts) is checked against the output of the Java implementation. The backend's unit tests include the trees used in the lesson.
3. Two group members who did not write an item review it against a checklist: aligned to its outcome, one clearly correct answer, each distractor tied to a misconception, feedback follows verdict–reason–next move, no more than about 60 words.
4. Wording is read aloud once to catch long or awkward sentences.

11.2 Example trees

	Tree A (worked example)	**Tree B (practice)**
Inserted in order	50, 30, 70, 20, 40, 60, 80	45, 25, 65, 15, 35, 55, 30
Shape	Perfect: every internal node has two children	Uneven: 35 has only a left child, 65 only a left child
Pre-order	50, 30, 20, 40, 70, 60, 80	45, 25, 15, 35, 30, 65, 55
In-order	20, 30, 40, 50, 60, 70, 80	15, 25, 30, 35, 45, 55, 65
Post-order	20, 40, 30, 60, 80, 70, 50	15, 30, 35, 25, 55, 65, 45
Search used	60: found after 6 visits	55: found after 7 visits

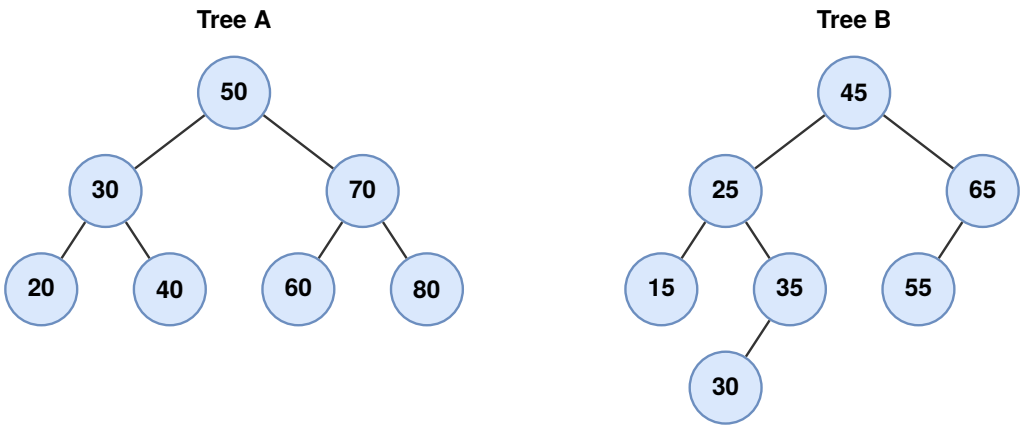


Figure 8: The two trees used in the lesson. Tree B's uneven shape stops students memorising a fixed pattern.

11.3 Trace of the practice search

Practice 1 asks students to predict each step of a DFS search for 55 on tree B. The expected answers, which the feedback is built on, are:

Step	Visit	What happens	Stack after (top first)
1	45	Not 55. Push 65, then 25.	25, 65
2	25	Not 55. Push 35, then 15.	15, 35, 65
3	15	Leaf. Backtrack.	35, 65
4	35	Not 55. Only a left child: push 30.	30, 65
5	30	Leaf. Backtrack.	65
6	65	Not 55. Only a left child: push 55.	55
7	55	Found. Stop.	(empty)

Step 4 is deliberately interesting: a node with only one child tests whether students really understand "skip empty children" or are just alternating left and right.

12 Implementation

12.1 Delivery to students

In a real rollout the lecturer would post the link on ClickUP two or three days before the lecture on tree traversal, with a one-line message: "Spend 20 minutes on this before Thursday's lecture. It is not marked." For the prototype the app runs locally (see the SE document), so delivery is limited to the peer pilot and the examination demonstration.

12.2 Guidance for the lecturer

- The app assumes no prior knowledge, so it can be set before the topic is introduced.
- The summary screen ends with two questions. Opening the lecture with them connects the lecture directly to what students did:

  1. "DFS needed six visits to find 60. Using the BST rule it needs only three. How, and what is the worst case?"

2. “When would you rather explore level by level instead of depth first?”

- The prototype records nothing. A later version could report anonymous, class-level quiz results so the lecturer can see which misconceptions are most common.

12.3 Examination demonstration

The ten-minute demonstration follows one student’s path through the app: hook, one concept screen, the worked example, a wrong prediction showing the hint ladder, a quiz question answered wrongly with confidence, and the guided review. This shows the instructional design working, not just the software.

13 Evaluation

13.1 Evaluation questions

1. Do students learn DFS from the app? (Kirkpatrick level 2, learning [13])
2. Do students find it usable and worth their time? (level 1, reaction)
3. Which screens or questions cause confusion?
4. Is the 4/6 threshold a sensible point at which to offer guided review?

Levels 3 and 4 (behaviour in the lecture and results in the module) cannot be measured within the examination timeline; Section 14 discusses how they could be measured later.

13.2 Formative evaluation plan

Following the staged approach of Dick and Carey’s formative evaluation, we evaluate in three rounds:

Round	When	What we do	Changes allowed
Expert review	4–5 Nov	A COS212 tutor checks the lesson text and quiz bank for accuracy and level.	Content corrections.
One-to-one	8 Nov	Two students who have not done COS212 use the app while thinking aloud; a group member observes and takes notes.	Wording, layout, hints.
Small group	9 Nov	Three to five students use the app independently and complete the instruments below.	Fixes only; no new features.

13.3 Instruments

- **Pre-test and post-test:** five short questions, in two parallel forms, taken before and after the session (Appendix D). Using different forms avoids simply measuring memory of the first test.
- **System Usability Scale (SUS):** the standard ten-item questionnaire [5].

- **Three open questions:** what helped most, what was confusing, what they would change.
- **Observation checklist:** where the student hesitates, uses hints, goes back, or says something unexpected.
- **Session time:** start and finish time.

13.4 Success criteria

The pilot is small, so these figures are indicators, not proof. We will consider the design to be working if:

- the average post-test score is at least two points higher than the pre-test score (out of five), and no student scores lower;
- the average SUS score is 68 or above (the commonly used benchmark for "above average");
- the median session time is between 15 and 30 minutes;
- no single screen is marked as confusing by more than one participant.

Checking the threshold. If students who scored 3/6 on the quiz do as well on the post-test as those who scored 4/6, the threshold is too high; if students who scored 4/6 still do poorly on the post-test, it is too low. With a small pilot this can only point in a direction, but it is better than leaving the number untested.

14 Reflection on our instructional choices

The specification asks us to reflect on our choices, not only to describe them. These are the decisions we debated most, and the trade-offs we accepted.

DFS on a tree rather than on a graph. On a tree, DFS does not need a "visited" set because there are no cycles. This makes the first encounter much simpler, but it means students do not yet meet the part of DFS that matters most for graphs. We think this is the right order: one new idea at a time. The lecture can add the visited set as the next step, and our summary questions point towards it.

Iterative stack first, recursion second. Showing the explicit stack makes backtracking visible, which directly addresses misconception M2. The cost is that the traversal orders are easiest to explain recursively, so students see two versions. We reduce the risk by showing the recursive call stack in the same stack panel and by saying explicitly that it is the same idea. We will watch for confusion here in the pilot.

Learning styles. We were asked about learning styles, and VARK is familiar to students and lecturers, so we used it to organise our thinking. But we chose not to build "style detection", because the evidence does not support it [17]. Giving everyone multiple representations is cheaper to build, better supported by research, and fairer.

No gamification. Points and badges would make a stronger first impression in a demo. We decided that time spent on precise, misconception-based feedback would do more for learning. This may make the app look plainer than some others; we accept that.

Offering, not forcing, remediation. A forced remediation loop guarantees that students see the review, but it treats them as if they cannot judge their own needs and can be frustrating. We chose to offer it, using the confidence rating to make the recommendation persuasive. Whether students actually take it up is something the pilot should tell us.

What the prototype cannot do yet. Without accounts or stored data we cannot offer spaced practice (coming back to the topic a few days later), which is one of the most reliable ways to improve long-term retention. We also cannot measure Kirkpatrick levels 3 and 4. Both would be natural next steps: a short follow-up quiz a week after the lecture, and anonymous class-level results for the lecturer.

References

[1] L. W. Anderson and D. R. Krathwohl (eds.). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[2] R. K. Atkinson, S. J. Derry, A. Renkl and D. Wortham. Learning from examples: instructional principles from the worked examples research. *Review of Educational Research*, 70(2):181–214, 2000.

[3] J. Biggs. Enhancing teaching through constructive alignment. *Higher Education*, 32(3):347–364, 1996.

[4] R. M. Branch. *Instructional Design: The ADDIE Approach*. Springer, 2009.

[5] J. Brooke. SUS: a "quick and dirty" usability scale. In P. W. Jordan et al. (eds.), *Usability Evaluation in Industry*, pp. 189–194. Taylor & Francis, 1996.

[6] CAST. *Universal Design for Learning Guidelines*, version 2.2. 2018. `http://udlguidelines.cast.org`

[7] A. N. Kumar, R. K. Raj, S. G. Aly, M. D. Anderson, B. A. Becker et al. *Computer Science Curricula 2023*. Association for Computing Machinery, New York, 2024.

[8] N. D. Fleming and C. Mills. Not another inventory, rather a catalyst for reflection. *To Improve the Academy*, 11:137–155, 1992.

[9] R. M. Gagné, W. W. Wager, K. C. Golas and J. M. Keller. *Principles of Instructional Design*, 5th ed. Wadsworth, 2005.

[10] J. Hattie and H. Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007.

[11] C. D. Hundhausen, S. A. Douglas and J. T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13(3):259–290, 2002.

[12] J. M. Keller. Development and use of the ARCS model of instructional design. *Journal of Instructional Development*, 10(3):2–10, 1987.

[13] D. L. Kirkpatrick and J. D. Kirkpatrick. *Evaluating Training Programs: The Four Levels*, 3rd ed. Berrett-Koehler, 2006.

[14] R. E. Mayer. *Multimedia Learning*, 2nd ed. Cambridge University Press, 2009.

[15] M. D. Merrill. First principles of instruction. *Educational Technology Research and Development*, 50(3):43–59, 2002.

[16] T. L. Naps, G. Rößling et al. Exploring the role of visualization and engagement in computer science education. *ACM SIGCSE Bulletin*, 35(2):131–152, 2003 (ITiCSE 2002 working group report).

[17] H. Pashler, M. McDaniel, D. Rohrer and R. Bjork. Learning styles: concepts and evidence. *Psychological Science in the Public Interest*, 9(3):105–119, 2008.

[18] V. J. Shute. Focus on formative feedback. *Review of Educational Research*, 78(1):153–189, 2008.

[19] J. Sweller. Cognitive load during problem solving: effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

[20] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[21] R. White and R. Gunstone. *Probing Understanding*. Falmer Press, 1992.

[22] D. Wood, J. S. Bruner and G. Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89–100, 1976.

A Lesson script

This is the text the student sees, screen by screen. Words in bold link to the glossary.

Screen 1: Hook (Event 1)

You are standing at the entrance of a maze. You can only see the passage right in front of you. At each fork you must choose a way to go.

How would you make sure you never miss the exit?
( ) Pick a random passage every time.
( ) Always take the left-most passage you have not tried yet. If you hit a dead end, go back to the last fork and try the next passage.
( ) Try the first step of every passage, then the second step of every passage, and so on.

After any choice: All three are real strategies, and two of them always work. Today you will learn the second one. It is called **depth-first search**.

Screen 2: Goals (Event 2)

In the next 20 minutes you will learn to:
1. Walk through a tree the depth-first way, and explain what that means.
2. Use a **stack** to keep track of where to go back to.
3. Check whether you are ready for the lecture, and get help if you are not.

You can go back to any screen at any time. Nothing here is marked.

Screen 3: Warm-up check (Event 3)

Question 1. Which of these pictures is a tree? *(three diagrams: a tree, a cycle, two separate pieces)*
Question 2. You push 1, then 2, then 3 onto a stack. Which number comes off first?

Refresher card: trees. A **tree** is made of **nodes** joined by lines. The top node is the **root**. Each node can have **children** below it. A node with no children is a **leaf**. Any node together with everything below it forms a **subtree**.

Refresher card: stacks. A **stack** is like a pile of plates. You can only put a plate on top (**push**) or take the top plate off (**pop**). The last plate you put on is the first one you take off.

Screen 4a: Binary search trees

In a **binary search tree** (BST), every node has at most two children: a left child and a right child. The tree follows one rule:

Smaller values go to the left. Larger values go to the right.

Try it: insert 65 into the tree below and watch where it goes.

Screen 4b: The big idea

Remember the maze? Depth-first search does the same thing in a tree.
Start at the root. Go down to the left as far as you can. When you reach a leaf (a dead end), go back up to the last node that still has an unexplored child, and go down that way.

Going back up is called **backtracking**.

Screen 4c: How DFS remembers

To backtrack, DFS needs to remember which nodes it still has to explore. It keeps them on a **stack**. When DFS leaves a node, it pushes the node's children onto the stack: **right child first, then left child**. Because the left child went on last, it is on top, so it comes off first. That is why DFS always explores the left side first.

Screen 4d: The four steps

1. Push the root onto the stack.
2. Pop the top node and **visit** it.
3. If it is the value you are looking for, stop. Otherwise push its right child, then its left child. Skip any child that is empty.
4. If the stack is empty, stop: the value is not in the tree. Otherwise, go back to step 2.

Screen 4e: Three orders, one walk

You can also use DFS to list every value in a tree. The walk is always the same. What changes is *when* you write a node down:
Pre-order: when you first arrive at the node.
In-order: after its left side, before its right side.
Post-order: when you leave the node for the last time.

Run in-order on this tree. Notice anything about the result? *(Reveal:)* It is sorted! In a BST, everything on the left is smaller and everything on the right is larger, so "left, me, right" always lists the values from smallest to largest.

Screen 4f: What DFS does not do

DFS does not look at the values to decide where to go. Even if you are searching for 80, and 80 is bigger than the root, DFS still explores the whole left side first.

That might seem wasteful. Keep the thought; we come back to it at the end.

Screen 5: Worked example (Event 5)

Let us search for 60. Click *Next* to take one step at a time.
Step 1. Visit 50. It is not 60. Push 70 (right), then 30 (left). 30 is on top.
Step 2. Visit 30. Not 60. Push 40, then 20. 20 is on top.
Step 3. Visit 20. Not 60. It is a leaf, so there is nothing to push. Backtrack: the top of the stack is now 40.
Step 4. Visit 40. Not 60. Another leaf. Backtrack: the top is now 70.

Step 5. Visit 70. Not 60. Push 80, then 60. 60 is on top.
Step 6. Visit 60. Found it! DFS visited 6 nodes.

Screen 6: Guided practice (Events 6 and 7)

Practice 1. Here is a new tree. This time, *you* say which node comes next. We are searching for 55. *(The student clicks a node before each step is revealed; the hint ladder applies.)*

Practice 2. Now on your own: drag the nodes of this tree into in-order. Hint: the answer should be sorted.

Screen 7: Readiness quiz (Event 8)

Six questions to check you are ready for the lecture. After each one, tell us how sure you were. There are no marks; this is just for you.

Screen 8: Summary (Event 9)

What you can now do
– Walk a tree depth-first: go deep on the left, backtrack, then go right.
– Use a stack to remember where to go back to (push right, then left).
– List a tree in pre-order, in-order and post-order, and know that in-order is sorted for a BST.

Take these to the lecture
1. DFS needed 6 visits to find 60. If you used the BST rule (smaller left, larger right), you would need only 3. How would that search work? What is the worst case?
2. When might you want to explore a tree level by level instead of depth first?

Try it yourself: build your own tree in the sandbox and predict the DFS order before you run it.

B Quiz bank

Tree A and tree B are shown in Figure 8. A tick (✓) marks the correct option. Each distractor is labelled with the misconception it targets.

Slot 1 (LO1, Remember)

1A What does DFS use to remember which nodes it still has to explore?

Option	Feedback
Stack ✓	Correct. DFS always continues from the node it saw most recently, and a stack gives back the most recent item first.
Queue (M1)	Not quite. A queue gives back the *oldest* item first, which explores the tree level by level. That is breadth-first search. DFS needs the newest item first: a stack.
Sorted list (M4)	Not quite. DFS does not use the order of the values at all. It needs to remember *where to go back to*, which a stack does.
Nothing; it just follows the lines	Not quite. After a leaf, DFS has to know where to go back to. Something must remember that: a stack.

1B You insert 65 into tree A. Where does it go?

Option	Feedback
Right child of 60 ✓	Correct. $65 > 50$: go right. $65 < 70$: go left to 60. $65 > 60$: go right, which is empty, so 65 goes there.
Left child of 70	Close, but 70 already has a left child (60). Keep comparing until you reach an empty spot.
Right child of 80	Not quite. 65 is smaller than 70, so at 70 you go left, not right.
Left child of 60	Not quite. 65 is larger than 60, so it goes to the right of 60.

Slot 2 (LO2, Understand)

2A DFS (pre-order) has just visited 30 in tree A. Click the node it visits next.

Option	Feedback
20 ✓	Correct. DFS goes as deep as possible first, starting with the left child.
70 (M1)	Not quite. 70 is on the same level as 30, so that would be moving across. Depth-first goes down first.
40 (M5)	Not quite. 40 is 30's right child. DFS pushes right first, then left, so the left child 20 is on top and comes off first.
50	Not quite. DFS only goes back up (backtracks) when it reaches a leaf. 30 still has children to explore.

2B Which sentence best describes backtracking in DFS?

Option	Feedback
Going back to the most recent node that still has an unexplored child ✓	Correct. That is exactly what popping the next node off the stack achieves.
Starting again from the root	Not quite. DFS never restarts. It continues from the most recent point that still has something left to explore.
Visiting the nodes in reverse order	Not quite. Backtracking is about where to go *next* after a dead end, not about reversing the order.
Moving to the next node on the same level (M1)	Not quite. That describes breadth-first search. In DFS, backtracking goes back *up* to a node you have already passed.

Slot 3 (LO3, Apply)

3A Put the nodes of tree A in pre-order.

Option	Feedback
50, 30, 20, 40, 70, 60, 80 ✓	Correct. Each node is written down when you first arrive, before its children.
20, 30, 40, 50, 60, 70, 80 (M3)	That is in-order. In pre-order a node comes *before* its children, so the root 50 comes first.
20, 40, 30, 60, 80, 70, 50 (M3)	That is post-order. In pre-order the root comes first, not last.
50, 30, 70, 20, 40, 60, 80 (M1)	That is level by level (breadth-first). Pre-order finishes the whole left subtree before moving to 70.

Ordering task. Feedback is chosen by the closest matching wrong sequence; any other wrong sequence gets general feedback that highlights the first position that is wrong.

3B Put the nodes of tree B in in-order.

Option	Feedback
15, 25, 30, 35, 45, 55, 65 ✓	Correct, and sorted, as in-order of a BST always is.
45, 25, 15, 35, 30, 65, 55 (M3)	That is pre-order. In in-order you write a node down *between* its left and right subtrees.
15, 25, 35, 30, 45, 55, 65	Very close. Look at 35 and 30: 30 is 35's left child, so it must come before 35.
15, 30, 35, 25, 55, 65, 45 (M3)	That is post-order. Hint: for a BST, the in-order is always sorted.

Ordering task, as for 3A.

Slot 4 (LO4, Apply)

4A During a DFS search on tree A the stack is [40, 70], top first. Which node is popped next?

Option	Feedback
40 ✓	Correct. The top of the stack is written first. 40 was pushed after 70, so it comes off first.
70 (M5)	Not quite. 70 is at the bottom. A stack gives back the last item pushed, which is 40.
20	Not quite. 20 is no longer on the stack; it has already been visited.
50	Not quite. The root was the first node visited and is no longer on the stack.

4B DFS is searching tree A for 60 and has just visited 70. What is on the stack now (top first)?

Option	Feedback
60, 80 ✓	Correct. 80 (right) was pushed first, then 60 (left), so 60 is on top.
80, 60 (M5)	Not quite. Right is pushed first and left last, so the left child 60 ends up on top.
70, 60, 80	Not quite. 70 has just been visited, so it has already been popped off.
60	Not quite. Both children of 70 are pushed, not just the one we are looking for. DFS does not know where 60 is yet.

Slot 5 (LO5, Analyse)

5A How many nodes does DFS visit to find 80 in tree A?

Option	Feedback
7 ✓	Correct. DFS explores the whole left subtree (30, 20, 40) and then 70 and 60 before reaching 80. 80 is the last node visited.
3 (M4)	That would be right if DFS used the BST rule (50, 70, 80). Plain DFS does not look at values, so it explores the left side first.
2	Not quite. DFS always starts at the root and checks every node it passes on the way.
6	Close. Count again: 50, 30, 20, 40, 70, 60, 80.

5B How many nodes does DFS visit to find 30 in tree B?

Option	Feedback
5 ✓	Correct: 45, 25, 15, 35, 30.
4 (M4)	That is the BST-rule path (45, 25, 35, 30). DFS also visits 15, because it always goes left first.
7	Not quite. DFS stops as soon as it finds the target. It never reaches the right subtree.
3	Not quite. Count each node DFS visits, starting from the root: 45, 25, 15, 35, 30.

Slot 6 (LO5, Analyse)

6A Why is the in-order of a BST always sorted?

Option	Feedback
Everything left of a node is smaller and everything right is larger, and in-order writes left, then the node, then right ✓	Correct. That holds at every node, so the whole list comes out from smallest to largest.
Because DFS compares values as it goes (M4)	Not quite. DFS never compares values to decide where to go. The sorting comes from the shape of the BST.
It only happens for this particular tree	Not quite. Try building your own BST in the sandbox: in-order is sorted every time.
Because inserting a value sorts the tree	Not quite. Inserting places a value in the right position, but the tree itself is not a sorted list. In-order is what reads it out in order.

6B A classmate says: "On a BST, DFS goes right whenever the target is bigger." What is wrong?

Option	Feedback
DFS ignores the values and always goes left first; the classmate is describing a different search ✓	Correct. That rule describes BST search, which you will meet in the lecture. Plain DFS explores blindly, left first.
Nothing, it is correct (M4)	Not quite. That is a common mix-up. DFS does not use the ordering; it always goes left first.
DFS goes *left* when the target is bigger	Not quite. DFS does not compare the target to choose a direction at all.
DFS chooses a direction at random	Not quite. DFS is completely predictable: always left first, then right.

C Glossary

Term	Meaning (as shown in the app)
Node	One item in a tree, holding a value.
Root	The node at the top of the tree. Every search starts here.
Child, parent	A node directly below another is its child; the node above is its parent.
Leaf	A node with no children: a dead end.
Subtree	A node together with everything below it.
Binary search tree (BST)	A tree where each node has at most two children, smaller values are on the left and larger values are on the right.
Stack	A collection where you can only add to the top (push) and take from the top (pop). Last in, first out.
Push, pop	Put an item on top of a stack; take the top item off.
Depth-first search (DFS)	Exploring a tree by going as deep as possible along one path before backing up to try another.
Visit	The moment DFS takes a node off the stack and looks at it.
Backtracking	Going back to the most recent node that still has an unexplored child.
Traversal	Visiting every node in a tree once, in a set order.
Pre-order, in-order, post-order	The three depth-first orders: a node is written down before, between or after its subtrees.

D Evaluation instruments

Pre-test and post-test (parallel forms)

Form 1 uses a tree built from 40, 20, 60, 10, 30, 50, 70; Form 2 uses 35, 15, 55, 5, 25, 45, 65. Students receive one form before the session and the other after; the order is alternated between participants.

1. Which data structure does DFS use to remember where to go back to?
2. Write down the pre-order of the tree.
3. Write down the in-order of the tree.
4. DFS is searching for the right-most leaf. List the nodes it visits, in order.
5. In one sentence, explain what "depth-first" means.

Each answer scores 1 (correct) or 0. Question 5 is marked correct if it mentions going deep along one path before trying another.

Usability and open questions

The ten standard SUS items [5] on a five-point scale, followed by:

1. What helped you most to understand DFS?
2. Was anything confusing? Where?
3. If you could change one thing about the app, what would it be?

Observation checklist

Observation	Screen	Time	Note
Hesitates for more than 10 seconds			
Uses a hint (level 1 / 2 / 3)			
Goes back to a previous screen			
Says something unexpected or incorrect aloud			
Opens the glossary			

E Acknowledgement of generative AI use

The specification allows generative AI to be used for content and interface work, provided it is acknowledged with the prompt and the tool. We used the following tools. All AI-generated text was read, checked and edited by group members. Every factual claim about the algorithm (visit orders, stack contents, node counts, quiz answers) was checked against our own Java implementation, and every reference was checked against the original source before submission.

Tool	Date	Used for and prompt (verbatim)
ChatGPT (OpenAI)	[date]	First draft of the SE document (v1.0, BST search). *Prompt:* [team to insert the exact prompt used].
Claude Opus 5.5 (Anthropic), in the Claude Code desktop app	28 Oct 2026	Revision of the SE document to v1.1 (algorithm changed to DFS, diagrams redrawn in LaTeX/TikZ). The COS750 examination specification PDF and the v1.0 draft were provided as input. *Prompt:* "Look at the attached document and [v1.0 draft pasted here] Review, humanise and add information value to the above content. write that in a latex format. Fix the algorithm to DFS and also explain in simple terms having in mind concepts of educational software dev. Also review the diagrams and draw them as though they were drawn on draw.io os a similar professional app without having AI feel/look."
Claude Opus 5.5 (Anthropic), in the Claude Code desktop app	28 Oct 2026	First draft of this ID document, including the lesson script and quiz bank. *Prompt:* "now write in full in the separate ID document"